Kubernetes Pod Scheduling — Complete Interview-Ready Study Guide

Node Selector · Node Affinity · Taints & Tolerations

> How to use this guide: Every section builds on the one before it. Read straight through without skipping. The goal is not to memorize answers — it is to understand the *reasoning* so deeply that you can answer questions you have never seen before. That is what gets you hired.

╔══ PART 1 — NODE SELECTOR ══╗

SECTION 1.1 — What NodeSelector Is and Why It Exists

What Interviewers Expect

> "NodeSelector is the most direct way to tell Kubernetes where a pod should land. In practice, I use it when the requirement is binary — either the node has the right label or the pod doesn't run at all. The reason I reach for it first is its simplicity, but I always warn my team: if someone removes or renames that label without updating the manifest, pods will silently go Pending in production. I've seen that happen at 2am and it is not fun to debug."

Beginner Explanation

Start Here — The Problem NodeSelector Solves

A Kubernetes cluster has many servers called nodes. Without any scheduling rules, Kubernetes places pods on any available node randomly. That is fine for simple apps — but in real life you often have requirements like:

- *"This banking application must only run on our compliance-approved servers"*
- *"This database must run on nodes with fast SSD storage"*
- *"This GPU workload must only run on GPU-equipped machines"*

NodeSelector is the simplest tool to enforce these requirements. You attach a label (a key-value tag) to a node, then tell your pod: *"Only go to nodes that carry this label."*

The Mental Model — A Strict Bouncer

Think of each node as a nightclub and NodeSelector as the bouncer's rulebook. The pod shows up at the door. The bouncer checks: *"Does this node have the label the pod needs?"* If yes, the pod gets in. If no node in the entire cluster has that label — the pod stands outside forever, waiting. This is called Pending state.

This is a hard rule. No label = no entry. No exceptions.

Real-World Job Scenario

Your company runs an internet banking application. Compliance regulations require it to run exclusively on a dedicated set of servers inside a secure network zone. Those servers are labeled `node-name: ib-node`. You write a NodeSelector rule targeting that label. Even if the cluster has 50 other nodes sitting completely idle, the banking app will never touch them.

The NodeSelector Manifest — Every Line Explained

```
# ——— FILE: nodeselector.yml
————————————————————————————————————————

apiVersion: apps/v1          # Which version of the Kubernetes
API to use
kind: Deployment             # We are creating a Deployment
(manages pods for us)
metadata:
  name: ib-deployment        # The name of this deployment
  labels:
    app: bank                # Label on the Deployment object
itself
spec:
  replicas: 2                # Run exactly 2 pods at all times
  selector:
```

```
    matchLabels:
      app: bank               # This Deployment manages any pod
labeled app: bank
  template:                   # Everything below here is the pod
blueprint
    metadata:
      labels:
        app: bank             # Every pod created gets this
label
    spec:
      nodeSelector:           # ← THE SCHEDULING RULE
        node-name: ib-node    # Pod ONLY runs on nodes labeled
node-name: ib-node
      containers:
        - name: cont1                                    #
Container name
          image: reyadocker/internetbankingrepo:latest #
Docker image to run
```

Field-by-Field Breakdown

```
┌──────────────────────────────┬──────────────────────────────────────
──────────────┐
│  YAML FIELD                  │  WHAT IT DOES IN PLAIN
ENGLISH              │
├──────────────────────────────┼──────────────────────────────────────
──────────────┤
│  apiVersion: apps/v1         │  Tells Kubernetes which rule
set to use     │
│  kind: Deployment            │  Creates a self-healing pod
manager          │
│  metadata.name               │  Human-readable name for this
deployment    │
│  spec.replicas: 2            │  Keep exactly 2 copies of
this pod running  │
│  selector.matchLabels        │  Links Deployment to the pods
it manages    │
│  nodeSelector:               │  Opens the scheduling
constraint block      │
│  node-name: ib-node          │  The label key-value the node
MUST have     │
│  containers.image            │  The Docker image that runs
inside the pod  │
```

```
└──────────────────────────────────┘└──────────────────────────────────────
──────────────┘
```

Step-by-Step Lab Workflow

```
# STEP 1 — Deploy the manifest
kubectl apply -f nodeselector.yml

# STEP 2 — Check pod status
kubectl get pods
# If output shows STATUS: Pending → the label is missing from
all nodes

# STEP 3 — Investigate why the pod is Pending
kubectl describe pod ib-deployment-7784c9bfb5-8sl8d
# Read the EVENTS section at the bottom — this is where the
failure reason lives

# STEP 4 — List all nodes to find the correct node ID
kubectl get nodes

# STEP 5 — Add the missing label using the command line
kubectl label nodes i-043783332483bf9f8 node-name=ib-node

# ALTERNATIVE to Step 5 — Edit the node directly in the editor
kubectl edit node i-043783332483bf9f8
# Inside the editor, under the labels: section, add:
# node-name: ib-node

# STEP 6 — Verify pods are now running
kubectl get pods
# STATUS should now show: Running

# STEP 7 — Clean up after the lab
kubectl delete deploy ib-deployment
```

Reading the Failure Message — Fully Translated

When you run `kubectl describe pod` on a Pending pod, you will see this in the Events section:

```
Warning  FailedScheduling  2m40s  default-scheduler
```

```
0/3 nodes are available:
  1 node(s) had untolerated taint
{node-role.kubernetes.io/control-plane: }
  2 node(s) didn't match Pod's node affinity/selector.
  preemption: 0/3 nodes are available: 3 Preemption is not
helpful for scheduling.
```

What Each Line Actually Means

```
┌──────────────────────────────────────────────────────────────────────
──────────────────┐
│  ERROR MESSAGE — LINE BY LINE TRANSLATION
│
├──────────────────────────────────────┬──────────────────────────────
──────────────────┤
│  "0/3 nodes are available"           │  Scheduler checked all 3
nodes.            │
│                                      │  Every single one was
rejected.             │
├──────────────────────────────────────┼──────────────────────────────
──────────────────┤
│  "1 node had untolerated taint       │  The Control Plane node
has a built-in    │
│   control-plane"                     │  "Keep Out" sign for
normal apps.          │
│                                      │  This is normal and
expected behavior.    │
├──────────────────────────────────────┼──────────────────────────────
──────────────────┤
│  "2 node(s) didn't match             │  The 2 worker nodes don't
carry the label │
│   selector"                          │  node-name: ib-node. That
is the real     │
│                                      │  cause of the Pending
state.                │
├──────────────────────────────────────┼──────────────────────────────
──────────────────┤
│  "Preemption is not helpful"         │  Even evicting a running
pod wouldn't      │
│                                      │  fix the problem — the
label mismatch     │
│                                      │  would still block
scheduling.             │
└──────────────────────────────────────┴──────────────────────────────
──────────────────┘
```

The fix is always the same: identify which label is missing, add it to the correct node, and the pod schedules itself automatically within seconds.

Faculty Notes Explanation:

```
kind: Deployment
metadata:
  name: ib-deployment
  labels:
    app: bank
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bank
  template:
    metadata:
      labels:
        app: bank
    spec:
      nodeSelector:
        node-name: ib-node
      containers:
      - name: cont1
        image: reyadocker/internetbankingrepo:latest

-- kubectl apply -f nodeselector.yml

-- kubectl get pods   --- Pods are not getting created because
there is no label
   to the node called node-name: ib-node

-- kubectl describe pod ib-deployment-7784c9bfb5-8sl8d

    Warning  FailedScheduling  2m40s  default-scheduler  0/3
nodes are available:
    1 node(s) had untolerated taint
{node-role.kubernetes.io/control-plane: },
    2 node(s) didn't match Pod's node affinity/selector.
    preemption: 0/3 nodes are available: 3 Preemption is not
helpful for scheduling.
```

```
-- kubectl get nodes

-- kubectl edit node i-043783332483bf9f8
        under labels add --- node-name: ib-node / We can use
command also
        "kubectl label nodes i-043783332483bf9f8
node-name=ib-node"

-- kubectl get pods  -- Now pods are running

we are forcing kube-schedular to schedule pod on particular
node. Its hard match.
If it doesn't match, Kube scheduler will not schedule the pod
so pod will be in pending state

-- kubectl delete deploy ib-deployment
```

╔══════════════════════════════════════╗
║ PART 2 — NODE AFFINITY ║
╚══════════════════════════════════════╝

SECTION 2.1 — What Node Affinity Is and Why NodeSelector Is Not Enough

What Interviewers Expect

> "Node Affinity is what I reach for when nodeSelector becomes too blunt an instrument. The real power is in the Hard vs. Soft distinction. Required affinity behaves just like nodeSelector — if nothing matches, the pod stays Pending. But preferred affinity is what I use for high-availability deployments. It tells the scheduler: try your best to follow my preference, but never sacrifice uptime for it. In practice, that distinction alone has saved us from scheduling freezes during infrastructure changes."

Beginner Explanation

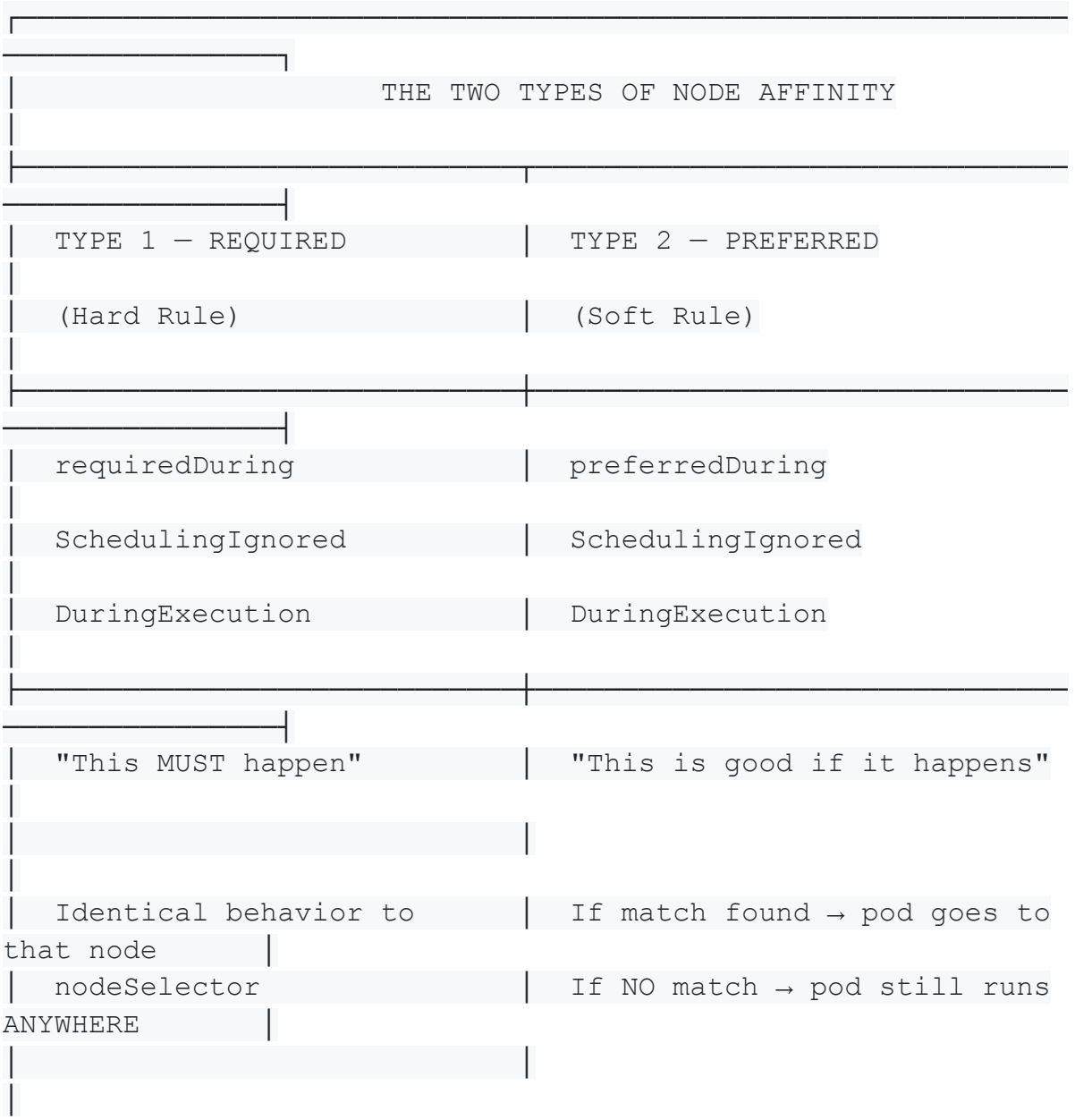
NodeSelector Has One Problem — It Is Too Rigid

NodeSelector only supports simple equality checks. Either the label matches exactly or the pod does not run. You cannot express things like:

- *"Put me on a node in Zone A OR Zone B"*
- *"Avoid nodes tagged as deprecated"*
- *"Prefer SSD nodes, but do not block me if none are available"*

Node Affinity solves all of this. It is `nodeSelector` with a brain — supporting logical operators, multiple conditions, and most importantly, the concept of soft vs. hard rules.

The Single Most Important Concept — Hard Rules vs. Soft Rules



```
│  No match = Pod PENDING      │  No match = Pod still RUNS
│
│  indefinitely                │  Never causes Pending state
│
├──────────────────────────────┼──────────────────────────────
──────────────┤
│  Use for: compliance,        │  Use for: performance
optimization,           │
│  licensed software,          │  zone preference, cost
optimization            │
│  strict hardware needs       │  with guaranteed uptime
│
└──────────────────────────────┴──────────────────────────────
──────────────┘
```

> What "IgnoredDuringExecution" means: The affinity rule only applies when the pod is first being placed. Once the pod is running, Kubernetes will NOT evict it — even if the node's labels change afterwards. The rule is evaluated once at scheduling time, then ignored forever.

The Logical Operators — Your Full Toolkit

```
┌──────────────────┬──────────────────────────────────┬──────────
──────────────────┐
│  OPERATOR        │  WHAT IT CHECKS                  │  REAL
USE CASE           │
├──────────────────┼──────────────────────────────────┼──────────
──────────────────┤
│  In              │  Value IS one of the listed      │
disktype In [ssd, nvme] │
│  NotIn           │  Value is NOT any of the listed  │  env
NotIn [dev, test]  │
│  Exists          │  The key exists (any value)      │  gpu
Exists             │
│  DoesNotExist    │  The key is completely absent    │
deprecated DoesNotExist │
│  Gt              │  Value is greater than number    │
cpu-cores Gt 8          │
│  Lt              │  Value is less than number       │
latency Lt 10           │
└──────────────────┴──────────────────────────────────┴──────────
──────────────────┘
```

Faculty Notes Explanation:

```
2: Node Affinity
================

Node Affinity is a more expressive way to specify rules about
the placement of
pods relative to nodes' labels. It allows you to specify rules
that apply only
if certain conditions are met. Same as Node Selector but this
has flexible
way, if matches do it, if not schedule pod on any another
node.

Two types
----------
1. Preferred during scheduling Ignore during execution (soft
rules) : good if happen
2. Required during scheduling Ignore during execution (hard
rules) : Must happen
   : Same as NodeSelector

The node affinity syntax supports the following operators:
In, NotIn, Exists, DoesNotExist, Gt, Lt, etc.
```

SECTION 2.2 — Required Affinity (Hard Rule) In Practice

What Interviewers Expect

"Required affinity is essentially nodeSelector with richer expression syntax. I use it when I absolutely need a pod on a specific type of node — for instance, a database that must have local NVMe storage. What most people miss is that required affinity can express complex conditions like 'the node must be in Zone A OR Zone B' — something nodeSelector simply cannot do. But the tradeoff is clear: if no node matches, your pod is Pending, and you need monitoring in place to catch that silently."

Beginner Explanation

Required affinity says: *"I will not run anywhere that does not meet my conditions."* The scheduler checks every node against your rules. If not a single

node qualifies, the pod waits in Pending state until a qualifying node appears or the rule is changed.

Real-World Scenario: You deploy a required affinity rule targeting nodes labeled `node-name: mb-node`. But your cluster has no nodes with that label. The pod goes Pending immediately. This is the expected behavior — it is telling you the infrastructure does not match your rule.

Required Affinity Manifest — Fully Annotated

```
# ——— FILE: required.yml
————————————————————————————————————————————————————

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ib-deployment
  labels:
    app: bank
spec:
  replicas: 1
  selector:
    matchLabels:
      app: bank
  template:
    metadata:
      labels:
        app: bank
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:  # ← HARD RULE
            nodeSelectorTerms:
              - matchExpressions:
                  - key: node-name       # Check this label key on every node
                    operator: In         # The value must be IN the list below
                    values:
                      - mb-node          # Node MUST have label node-name: mb-node
```

```
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

Step-by-Step Lab Workflow

```
# STEP 1 — Deploy the required affinity manifest
kubectl create -f required.yml

# STEP 2 — Check pod status and which node it is on
kubectl get pods -o wide
# Expected: STATUS = Pending
# Reason: required rule targets mb-node but no node has that
label

# STEP 3 — Clean up
kubectl delete -f required.yml
```

What This Lab Proves

The pod goes Pending because `required` affinity is a hard rule — identical in behavior to `nodeSelector`. The cluster had no node labeled `node-name: mb-node`, so the scheduler rejected all available nodes and left the pod waiting indefinitely.

Faculty Notes Explanation:

```
- matchExpressions:
  - key: node-name
    operator: In
    values:
    - mb-node
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80

-- kubectl create -f required.yml
```

```
-- kubectl get pods -o wide   [Pod are not running, it is in pending state because,
   required is mb-node, but no node has mb-node]

-- kubectl delete -f required.yml
```

SECTION 2.3 — Preferred Affinity (Soft Rule) In Practice

What Interviewers Expect

"Preferred affinity with weights is what I use when availability matters more than placement precision. The weight system is elegant — every node gets a score based on how many preference rules it satisfies, and the pod lands on the highest-scoring node. In practice, I combine multiple weighted preferences to create an intelligent priority ranking. The pod always runs. It just runs in the best place available at that moment — and that is exactly the behavior you want in a production cluster."

Beginner Explanation

The Analogy — A Hotel With Ranked Room Preferences

When you book a hotel, you say: *"I prefer a sea-view room. If unavailable, a garden-view is fine. If neither is available, any room will do."* You always get a room — you never sleep on the street.

Preferred affinity works exactly the same way. You give the scheduler a ranked wishlist. It finds the best available option. If nothing on the wishlist is available, the pod still runs — it lands on whatever node is available.

How the Weight Scoring System Works

Every node starts with a score of 0. For each Preferred rule a node satisfies, it gains `+weight` points. After scoring all nodes, the pod goes to the highest-scoring node.

```
SCORING EXAMPLE — Two nodes, one preference rule (weight: 1)

Node A  →  labeled node-name: mb-node  →  Matches rule  →  Score: +1  →  WINNER
```

```
Node B  →  no matching label          →  No match      →
Score:  0

Result: Pod schedules to Node A.
```

```
If Node A did not exist at all:

Node B  →  Score: 0  →  Still WINS (only option available)

Result: Pod runs on Node B. No Pending. No failure. App stays
up.
```

Real-World Job Scenario

You manage a banking application across two AWS availability zones. Zone A has lower database latency so you prefer it. But you cannot afford downtime if Zone A goes offline. You set Zone A as preferred (weight: 80) and Zone B as secondary (weight: 20). On a normal day, all pods land in Zone A. If Zone A goes offline entirely, the scheduler quietly starts placing pods in Zone B — no manual intervention, no downtime, no on-call alert at 3am.

What Actually Happened in the Classroom Lab

```
# The lab opened with two old pods Terminating from a previous
exercise
kubectl get pods
# NAME                                READY   STATUS
RESTARTS   AGE
# ib-deployment-7784c9bfb5-fv5qd      1/1     Terminating   0
8m4s
# ib-deployment-7784c9bfb5-s2hzp      1/1     Terminating   0
8m4s

# Open the preferred affinity manifest in the editor
vi prefered.yml

# Check what nodes are available before deploying
kubectl get nodes
```

```
# NAME                          STATUS   ROLES           AGE
VERSION
# i-004f9b5037e50bcfd           Ready    node            16m
v1.25.16
# i-02e9e62e21f186d3f           Ready    control-plane   18m
v1.25.16
# i-0e20e80d62aba4c2c           Ready    node            16m
v1.25.16
#
# 2 worker nodes available. Control-plane will not accept pods
(it has a taint).

# Deploy the manifest
kubectl apply -f prefered.yml
# Output confirms: deployment.apps/ib-deployment created

# Check where the pod actually landed — the NODE column shows
you
kubectl get pods -o wide
```

Preferred Affinity Manifest — Every Line Explained

```
# ─── FILE: preferred.yml
──────────────────────────────────────────────────────

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ib-deployment           # Name of this Deployment
  labels:
    app: bank                   # Label on the Deployment
object itself
spec:
  replicas: 1                   # Run 1 pod
  selector:
    matchLabels:
      app: bank                 # Manages pods labeled app:
bank
  template:                     # Blueprint for every pod this
Deployment creates
    metadata:
      labels:
        app: bank               # This label must match
selector.matchLabels above
```

```
    spec:
      affinity:                  # Opens the affinity
configuration block
        nodeAffinity:            # We are writing node-level
scheduling rules
          preferredDuringSchedulingIgnoredDuringExecution:   #
← SOFT RULE
          - weight: 1            # Score added to a node if it
matches this rule
            preference:          # The condition that earns this
weight
              matchExpressions:
              - key: node-name        # Check for the label
key: node-name
                operator: In         # Value must be IN the
list below
                values:
                - mb-node            # Prefer nodes labeled
node-name: mb-node
      containers:
      - name: nginx
        image: nginx:1.14.2          # The container image to
run
        ports:
        - containerPort: 80          # Port the container
listens on inside the pod
```

Field-by-Field Reference

```
┌──────────────────────────────────────────────────────────────────
────────────────┐
│  PREFERRED AFFINITY — COMPLETE FIELD REFERENCE
│
├──────────────────────────────────────────┬──────────────────────
────────────────┤
│  FIELD                                   │  MEANING
│
├──────────────────────────────────────────┼──────────────────────
────────────────┤
│  preferredDuringScheduling               │  Soft rule — "try to"
not "must"    │
│  IgnoredDuringExecution                  │  Rule ignored after
pod starts      │
```

```
├──────────────────────────────────────────┼──────────────────────────
──────────────┤
│  weight: 1                               │  Score value for this
rule.          │
│                                          │  Range: 1 to 100.
│
│                                          │  Higher = stronger
preference.       │
│                                          │  Multiple rules stack
their scores.  │
├──────────────────────────────────────────┼──────────────────────────
──────────────┤
│  preference.matchExpressions             │  The logical
condition to evaluate   │
├──────────────────────────────────────────┼──────────────────────────
──────────────┤
│  key: node-name                          │  The label key to
check on the node  │
├──────────────────────────────────────────┼──────────────────────────
──────────────┤
│  operator: In                            │  Logic: value must be
in the list    │
├──────────────────────────────────────────┼──────────────────────────
──────────────┤
│  values: [mb-node]                       │  The acceptable label
values         │
└──────────────────────────────────────────┴──────────────────────────
──────────────┘
```

> Key Insight on weight: 1 — A weight of 1 is deliberately low. In this lab, there is only one preference rule, so even a score of 1 is enough to make a node the winner. In production with multiple competing preference rules, you assign weights like 80, 50, and 20 to create a clear ranking system. The scheduler always picks the highest-scoring node.

Faculty Notes Explanation:

```
Preferred:
----------

vi preferred.yml

apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: ib-deployment
  labels:
    app: bank
spec:
  replicas: 1
  selector:
    matchLabels:
      app: bank
  template:
    metadata:
      labels:
        app: bank
    spec:
      affinity:
        nodeAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 1
            preference:
              matchExpressions:
              - key: node-name
                operator: In
                values:
                - mb-node
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

weight: 1: Indicates the preference weight. Higher values signify stronger preferences.

[Terminal lab — verbatim output from classroom:]

```
[root@kops ~]# kubectl get pods
NAME                                READY   STATUS
RESTARTS   AGE
ib-deployment-7784c9bfb5-fv5qd      1/1     Terminating   0
8m4s
ib-deployment-7784c9bfb5-s2hzp      1/1     Terminating   0
8m4s
```

```
[root@kops ~]# vi prefered.yml

[root@kops ~]# kubectl get nodes
NAME                          STATUS   ROLES           AGE
VERSION
i-004f9b5037e50bcfd           Ready    node            16m
v1.25.16
i-02e9e62e21f186d3f           Ready    control-plane   18m
v1.25.16
i-0e20e80d62aba4c2c           Ready    node            16m
v1.25.16

[root@kops ~]# kubectl apply -f prefered.yml
deployment.apps/ib-deployment created

[root@kops ~]# kubectl get pods -o wide
```

║ PART 3 — TAINTS AND TOLERATIONS ║

SECTION 3.1 — What Taints and Tolerations Are and Why They Exist

What Interviewers Expect

> "Taints and Tolerations are the inverse of Node Affinity. Affinity is the pod saying 'I want to go there.' A taint is the node saying 'you cannot come here unless you can tolerate me.' In practice, I use taints to dedicate nodes to specific workloads. GPU nodes get tainted so only GPU-aware pods land there. The Control Plane has a built-in taint — that is why normal apps never accidentally schedule on the master node. Understanding that taint is what let me explain a production Pending error in my first week on the job."

Beginner Explanation

The Problem Taints Solve

Imagine you have a Kubernetes cluster with 10 nodes. Three of those nodes have expensive GPU hardware, reserved exclusively for machine learning jobs. Without any protection, Kubernetes might place a simple web server pod on a GPU node — wasting expensive hardware on a workload that does not need it.

Taints are the solution. A taint is a *"Keep Out"* sign you place on a node. It repels all pods that do not explicitly declare they can tolerate that taint.

Tolerations are the *"permission slips"* you attach to pods. A pod with a matching toleration is allowed past the taint. A pod without a toleration is rejected from that node entirely.

The Analogy — Allergen Warning + Declaration of Safety

Think of a taint like a restaurant posting: *"This kitchen uses peanuts."* Most customers (pods) will avoid it by default. But a customer who explicitly declares *"I am not allergic to peanuts"* (a toleration) can eat there safely.

How This Connects to What You Already Know

You already saw taints in action back in Section 1.1, inside the error message:

```
1 node(s) had untolerated taint
{node-role.kubernetes.io/control-plane: }
```

That was the Control Plane node rejecting your pod. The Control Plane has a built-in `NoSchedule` taint applied automatically by Kubernetes. Your pod had no toleration for it, so it was blocked. This is intentional — master nodes run cluster management components and must not be cluttered with application pods.

The Three Taint Effects

```
|  NoSchedule            |  Pods WITHOUT a matching toleration
will NEVER be     |
|                        |  scheduled on this node.
|
|                        |  Pods already running here are NOT
affected.          |
├────────────────────────┼──────────────────────────────────────
──────────────────┤
|  PreferNoSchedule      |  Kubernetes will TRY to avoid
scheduling pods here,   |
|                        |  but will do so if no other node is
available.         |
|                        |  This is the soft version of
NoSchedule.                |
├────────────────────────┼──────────────────────────────────────
──────────────────┤
|  NoExecute             |  Pods without a toleration will NOT
be scheduled      |
|                        |  here AND existing pods already
running here will     |
|                        |  be immediately EVICTED.
|
└────────────────────────┴──────────────────────────────────────
──────────────────┘
```

Faculty Notes Explanation:

```
TAINTS and TOLERANCE
====================

In Kubernetes, taints and tolerations work together to control
the scheduling
of Pods onto Nodes. Taints are applied to Nodes to prevent
certain Pods from
being scheduled on them, while tolerations are applied to Pods
to allow them
to be scheduled on Nodes with matching taints.

Example Scenario: Dedicated Nodes for Specific Workloads
--------------------------------------------------------

Suppose you have a Kubernetes cluster with Nodes equipped with
specialized
```

```
hardware, such as GPUs, intended exclusively for machine
learning workloads.
To ensure that only Pods requiring GPU resources are scheduled
on these
Nodes, you can use taints and tolerations.

-- kubectl edit node i-0e6b67bf1b00383be  [Edit control plane,
this master node
   has taint no schedule, thats the reason master node will
not have pods]
```

SECTION 3.2 — Applying a Taint to a Node

What Interviewers Expect

"Applying a taint is a single kubectl command but the syntax needs to be second nature in an interview. The format is always key=value:effect. The key and value describe the nature of the restriction, and the effect tells the scheduler exactly what to do with non-tolerating pods. In practice, I always verify the taint was applied correctly by running kubectl describe node and checking the Taints field at the top — a typo in the taint is completely silent and causes confusing scheduling behavior."

Beginner Explanation

Applying a taint is like installing a lock on a room. Once the lock is on, only people holding the right key (toleration) are allowed inside. Everyone else is turned away at the door.

The Taint Command — Fully Dissected

```
kubectl taint nodes i-0333b24c25bf4868b
hardware=gpu:NoSchedule
```

```
┌──────────────────────────────────────────────────────────────
──────────────────┐
│  TAINT COMMAND SYNTAX BREAKDOWN
│
├──────────────────────┬──────────────────────────────────────
──────────────────┤
```

```
|  kubectl taint    |  The taint subcommand
|
|  nodes            |  We are tainting a node (not a pod)
|
|  i-0333b24...     |  The specific node to apply the taint to
|
|  hardware         |  KEY — the taint identifier/category
|
|  gpu              |  VALUE — describes what kind of taint
this is              |
|  NoSchedule       |  EFFECT — what happens to non-tolerating
pods                 |
└──────────────────┴──────────────────────────────────────────
──────────────────┘
```

```
Full syntax:  kubectl taint nodes [NODE-NAME]
[KEY]=[VALUE]:[EFFECT]
```

What This Command Does

Once applied, any pod that does NOT declare a toleration for `hardware=gpu` will be immediately rejected from this node. Only pods explicitly carrying a matching toleration will be considered for scheduling here.

Verify the Taint Was Applied Correctly

```
# Check the node to confirm the taint appears
kubectl describe node i-0333b24c25bf4868b

# Look for this in the output:
# Taints:   hardware=gpu:NoSchedule

# List all nodes to find node names first
kubectl get nodes
```

Faculty Notes Explanation:

```
Apply a Taint to GPU Nodes
--------------------------

First, taint the GPU Nodes to repel/force Pods that do not
require GPU resources:
```

```
-- kubectl get nodes

-- kubectl taint nodes i-0333b24c25bf4868b
hardware=gpu:NoSchedule

This command adds a taint with key hardware, value gpu, and
effect NoSchedule
to the specified Node. As a result, Pods without a matching
toleration will
not be scheduled on this Node.

Lets test:
--------

vi deploy.yml

apiVersion: apps/v1
kind: Deployment
metadata:
```

SECTION 3.3 — Writing Tolerations in the Pod Manifest

What Interviewers Expect

"A toleration in the pod spec is the pod declaring 'I acknowledge this taint and I am permitted to run on that node.' The critical thing to understand — and what most people get wrong — is that a toleration is permission, not a guarantee. Just because a pod tolerates a taint does not mean the scheduler will definitely send it to the tainted node. To guarantee placement on a specific tainted node, you combine a toleration with a nodeSelector or nodeAffinity rule. That combination is the production pattern for dedicated GPU or high-memory node pools."

Beginner Explanation

Toleration Is Permission, Not a Guarantee — This Is Critical

This is the most common misunderstanding about Taints and Tolerations:

```
┌──────────────────────────────────────────────────────────────────────
──────────────┐
```

```
│  WHAT A TOLERATION DOES vs. WHAT IT DOES NOT DO
│
├──────────────────────────────────────┬──────────────────────────────────────
──────────────┤
│  WHAT IT DOES                        │  WHAT IT DOES NOT DO
│
├──────────────────────────────────────┼──────────────────────────────────────
──────────────┤
│  Allows the pod to be scheduled      │  Does NOT force the pod
to go there       │
│  on a node with that taint           │
│
├──────────────────────────────────────┼──────────────────────────────────────
──────────────┤
│  Removes the rejection barrier       │  Does NOT act as an
attraction — the      │
│  for that specific node              │  pod can still go
anywhere else           │
└──────────────────────────────────────┴──────────────────────────────────────
──────────────┘

To FORCE a pod to a specific tainted node:
Toleration (permission) + NodeAffinity/NodeSelector
(attraction) = Guaranteed placement
```

The Complete Test Manifest — Toleration Without Affinity

```
# ─── FILE: deploy.yml
──────────────────────────────────────────────────────────────

apiVersion: apps/v1
kind: Deployment
metadata:
  name: gpu-deployment
  labels:
    app: ml-workload
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ml-workload
  template:
    metadata:
      labels:
```

```
        app: ml-workload
    spec:
      tolerations:                    # ← THE PERMISSION SLIP
BLOCK
        - key: "hardware"             # Must match the taint key
exactly
          operator: "Equal"           # Equal = check key AND
value
          value: "gpu"                # Must match the taint
value exactly
          effect: "NoSchedule"        # Must match the taint
effect exactly
      containers:
        - name: gpu-container
          image: nvidia/cuda:11.0-base
```

Toleration Field-by-Field Reference

```
┌──────────────────────────────────────────────────────────────────
──────────────┐
│  TOLERATION FIELDS — EVERY FIELD MUST MATCH THE TAINT
│
├────────────────────┬─────────────────────────────────────────────
──────────────┤
│  FIELD             │  EXPLANATION
│
├────────────────────┼─────────────────────────────────────────────
──────────────┤
│  key               │  Must match the taint's key exactly
(hardware)            │
├────────────────────┼─────────────────────────────────────────────
──────────────┤
│  operator          │  "Equal" → matches key AND value
│
│                    │  "Exists" → matches key regardless of
value               │
├────────────────────┼─────────────────────────────────────────────
──────────────┤
│  value             │  Must match the taint's value exactly
(gpu)               │
│                    │  Not required when operator is "Exists"
│
├────────────────────┼─────────────────────────────────────────────
──────────────┤
```

```
|  effect            |  Must match the taint's effect exactly
(NoSchedule)      |
|                    |  If omitted, the toleration matches ALL
effects           |
└────────────────────┴──────────────────────────────────────────
──────────────┘
```

The Production Pattern — Toleration + Affinity Together

When you need a pod to land exclusively on a specific tainted node, combine both tools:

```
spec:
  tolerations:
    - key: "hardware"
      operator: "Equal"
      value: "gpu"
      effect: "NoSchedule"
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: hardware
                operator: In
                values:
                  - gpu
```

```
Toleration alone    → Pod CAN go to GPU node but might go
elsewhere
Affinity alone      → Pod WANTS to go to GPU node but taint
blocks it
Toleration + Affinity → Pod goes to GPU node. Guaranteed.
Every time.
```

Faculty Notes Explanation:

```
Suppose you have a Kubernetes cluster with Nodes equipped with
specialized
hardware, such as GPUs, intended exclusively for machine
learning workloads.
```

```
To ensure that only Pods requiring GPU resources are scheduled
on these
Nodes, you can use taints and tolerations.

-- kubectl edit node i-0e6b67bf1b00383be  [Edit control plane,
this master node
   has taint no schedule, thats the reason master node will
not have pods]

Apply a Taint to GPU Nodes
--------------------------

First, taint the GPU Nodes to repel/force Pods that do not
require GPU resources:

-- kubectl get nodes

-- kubectl taint nodes i-0333b24c25bf4868b
hardware=gpu:NoSchedule

This command adds a taint with key hardware, value gpu, and
effect NoSchedule
to the specified Node. As a result, Pods without a matching
toleration will
not be scheduled on this Node.

Lets test:
--------

vi deploy.yml

apiVersion: apps/v1
kind: Deployment
metadata:
  [name: gpu-deployment — reconstructed from context, image
visible in screenshot]
```

‖ MASTER REFERENCE — Everything on One Page ‖

Concept Decision Tree — Which Tool Do I Use?

```
You need to control WHERE a pod runs
                  |
                  ▼
    Does the requirement come from
    the POD side or the NODE side?
            |                |
         POD SIDE        NODE SIDE
      (pod requests     (node repels
       a location)       certain pods)
            |                |
            ▼                ▼
    Is the rule       Use TAINTS
    absolute or       on the node
    flexible?         + TOLERATIONS
       |        |       on the pod
   ABSOLUTE FLEXIBLE
   (must)   (prefer)
      |        |
      ▼        ▼
  Required  Preferred
  Affinity  Affinity
  (or       (with
  NodeSel.) weights)
      |        |
      ▼        ▼
   Pending  Always
   if no    runs —
   match    best node
   found    wins
```

Complete Command Reference — Every Command From the Faculty Notes

```
# ── NODE OPERATIONS
──────────────────────────────────────────────────────────────
kubectl get nodes                                    # List all
nodes in cluster
kubectl get nodes --show-labels                      # List
nodes with labels
```

```
kubectl describe node <node-name>                          # Full
details + Taints field
kubectl edit node <node-name>                              # Open
node in editor
kubectl label nodes <node-name> key=value                  # Add a
label via command line
kubectl label nodes i-043783332483bf9f8 node-name=ib-node   #
Exact lab command

# ── TAINT OPERATIONS
──────────────────────────────────────────────────────────────

kubectl taint nodes <node-name> key=value:effect    # Apply a
taint
kubectl taint nodes i-0333b24c25bf4868b
hardware=gpu:NoSchedule  # Exact lab command
kubectl taint nodes <node-name> key=value:effect-   # Remove a
taint (trailing -)

# ── DEPLOYMENT OPERATIONS
──────────────────────────────────────────────────────

kubectl apply -f <filename>.yml                            # Create
or update from file
kubectl create -f <filename>.yml                           # Create
only (fails if exists)
kubectl delete -f <filename>.yml                           # Delete
everything in the file
kubectl delete deploy <deployment-name>                    # Delete a
named deployment
kubectl delete deploy ib-deployment                        # Exact
lab command

# ── POD INSPECTION
──────────────────────────────────────────────────────────

kubectl get pods                                           # Basic
pod status
kubectl get pods -o wide                                   # Pod
status + which node it is on
kubectl describe pod <pod-name>                            # Full
details + Events section
kubectl describe pod ib-deployment-7784c9bfb5-8sl8d # Exact
lab command
```

All Three Tools Compared Side by Side

	NODE SELECTOR	NODE AFFINITY	TAINTS & TOLERATIONS
Direction	Pod → Node	Pod → Node	Node repels Pod
	Pod requests	Pod requests	Pod must tolerate
Hard/Soft	Hard only	Required = Hard	NoSchedule = Hard
		Preferred = Soft	PreferNo = Soft
			NoExecute = Evict
Logic support	Exact match only	In, NotIn,	Equal, Exists
		Exists, Gt, Lt	
No match result	Pod → Pending	Required→Pending	Pod rejected from
		Preferred→Runs	tainted nodes only
Best used for	Simple fixed	Flexible HA	Dedicated nodes,
	placement rules	aware placement	hardware isolation

The Interview Answer That Ties Everything Together

If an interviewer asks: *"How do you control pod placement in Kubernetes?"*

"There are three tools and knowing when to reach for each one is the real skill. For simple non-negotiable placement — like a licensed app that must stay on a specific node — I use `nodeSelector` or required Node Affinity. Both block the pod from running if the label is absent, which is exactly the right failure behavior for compliance-driven requirements.

For production services where uptime matters more than perfect placement, I use preferred Node Affinity with weights. The scheduler follows my preferences when it can but never sacrifices availability to do so. Multiple weighted rules let me build an intelligent priority ranking — primary zone at weight 80, fallback zone at weight 20. The pod always runs. It just always runs in the best place available.

Taints and Tolerations are the third dimension and they work from the opposite direction. Instead of the pod requesting a location, the node is repelling unwanted workloads. I taint GPU nodes with `hardware=gpu:NoSchedule` so only ML workloads that explicitly carry a matching toleration can land there. Everything else bounces off automatically. The Control Plane uses exactly this mechanism — that is why application pods never accidentally end up on the master node.

The most powerful production pattern combines all three — a taint protects the node, a toleration grants the pod permission, and a nodeAffinity rule actively attracts the pod there. That combination gives you complete bidirectional control over scheduling without any ambiguity."

End of Study Guide — 3 Parts · 7 Sections · All Faculty Commands Included · All Faculty Manifests Included